Name: Nguyễn Hoàng Phú Tài
MSSV: N25DCCE084
Class: E25CQCE02-N

Problem 1: 2,25
Problem 2: 2,0
Problem 3: 2,5
Problem 4: 2,5
9,25

problem 1: 2,25

Given a singly linked list: head → [10] → [25] → [40] → [15] → Null

+) Task 1: Write a function in pseudocode to reverse the singly linked list, including three pointers (prev, curr, next) to complete your function

+) Task 2: Draw a diagram or step-by-step pointer trace table showing the positions and values of the three pointers (prev, curr, next) after each iteration until the list is fully reversed.

Solve Problem 1

```
Algorithm ReverseLinkedList(head):
    prev ← NULL
    curr ← head
    next ← NULL
    while (curr is not NULL do:)
        next ← curr.next
        curr.next ← prev
        prev ← curr
        curr ← next
    end while
    ~~return head~~
    return prev
```

Vibook
"Giữa thành công và thất bại có con sông gian khổ..trên con sông đó có cây cầu tên là sự cố gắng".
Scanned with CamScanner

1.25 Thứ ngày tháng năm

Iteration	Prev	curr	next	linked list
i = 0	Null	10	Null	10 -> 25 -> 40 -> 15 -> Null
i = 1	10	25	25	10 -> Null 25 -> 40 -> 15 -> Null
i = 2	25	40	40	25 -> 10 -> Null 40 -> 15 -> Null
i = 3	40	15	15	40 -> 25 -> 10 -> Null 15 -> Null
i = 4	15	Null	Null	15 -> 40 -> 25 -> 10 -> Null

Problem 2: 2,00

Given a singly circular linked list: head -> [10] -> [20] -> [30] -> head

0,75 Task 1: Write pseudocode for the function InsertHead (head, newValue) to insert a new node containing newValue at the front of the circular linked list while maintaining its circular structure.

- Task 2: Trace the execution steps (via text description or pointer diagrams) when inserting newValue = 5 into the given list.

```
Algorithm InsertHead (head, newValue):
    newNode ← CreateNode (newValue)
    if head is Null then
        newNode.next ← newNode
        head ← newNode
        return head
    end if
```

Vibook
"... bi quan để cơ hội chìm vào khó khăn, người lạc quan biến khó khăn thành cơ hội".
Scanned with CamScanner

Thứ ngày tháng năm

tail ← head
(while tail.next ≠ head) -0.5
    tail ← tail.next
    ↳ lặp vô tận!
end while

newNode.next ← head
tail.next ← newNode
head ← newNode
return head

1.25 Task2: Initial State
head -> [10]
[30] tail -> [10] head

Step 1: Create a new node: newNode = 5
Step 2: Traverse to find the last node (tail)
Start in tail = head [10] and go traverse all to come tail.next is 10 (head). That means tail is 30.
Step 3: Update newNode.next
newNode.next = head
=> [5] -> [10]
Step 4: Update tail.next
tail.next = newNode
=> [30] -> [5]
Step 5: Update head
head = newNode
=> head -> [5]

We get final State
head -> [5] ->
[10] -> [20] -> [30]
-> head [5]

Vibook
"Thứ khiến người ta phát ra ánh sáng không phải là châu ngọc trên quần áo mà là tri thức sâu trong tâm hồn".
Scanned with CamScanner

Problem 3: Doubly linked list

- Task 1: write pseudocode for InsertBefore (head, target, newValue) to insert a new node containing newValue immediately before the first occurance of target
- Task 2: List the exact order of pointer assignments required and briefly justify why this specific sequence avoids memory leaks or broken links.
- Task 3: Explicitly handle the following two edge cases in code
  1. target is located at the head node (head)
  2. target does not exist in the list

```
Algorithm InsertBefore (head, target, newValue):
    if head is Null then
        print "List is empty, target not found"
        return head
    end if
    curr ← head
    while curr is not Null and curr.data != target do
        curr ← curr.next
    end while
    if curr is Null then
        print "Target value not found in the list"
        return head
    end if
    newNode ← CreateNode (newValue)
```

Vibook
"Sống có mơ ước nhưng hãy để đôi chân mình bước đi trên hiện thực"
Scanned with CamScanner

Name: Nguyễn Hoàng Phú Tài

MSSV: N25DECE089 Thứ Ngày tháng năm

class E25CQCE02-N

```
if curr == head then
    newNode.next ← head
    newNode.prev ← Null
    head.prev ← newNode
    head ← newNode
    return head
end if
newNode.next ← curr
newNode.prev ← curr.prev
curr.prev.next ← newNode
curr.prev ← newNode
return head
```

0,5 Task2: If we want insert newNode between A and B

newNode.next = B (right hand of newNode catch B)

newNode.prev = A (Left hand of newNode catch A)

A.next = newNode (A let go B to catch newNode)

B.prev = newNode (B let go A to catch newNode

In this case B is curr and A is curr.prev

Detailed Justification:

Safety First: Assigning like Step 1 and 2 connects the new node to existing adjacent nodes without overwriting or severing any.

Vibook "Giữa thành công và thất bại có con sông gian khổ, trên con sông đó có cây cầu tên là sự cố gắng".

Scanned with CamScanner

existing pointers in the original list.

Preventing Broken Links / Memory Leaks

Step 3 befor Step 4 would cause us to lose the reference to the original predecessor ~~so~~ node, resulting in a broken list structure and a potential memory ~~be~~ leak

Completing the connection:

Once curr_prev_next is updated to point to newNode, it is finally safe to update curr_prev to point back to newNode, cleanly integrating NewNode into the doubly linked lisk

40 Task 3:

target is located at the head node

```
if curr == head then
    newNode.next ← head
    newNode.prev ← Null
    head.prev ← newNode
    head ← newNode
    return head
end if
```

Vibook

"Người bi quan để cơ hội chìm vào khó khăn, người lạc quan biến khó khăn thành cơ hội".

Scanned with CamScanner

Thứ ngày tháng năm

target does not exist in the list
if curr is Null then
print "Error: Target value does not exist."
return head
end if

problem 4: 2.5

Given a singly linked list sorted in ascending order: head -> 2 -> 5 -> 8 -> Null

- Task 1: Write the function SortedInsert(head, val) to insert val into the list such that the ascending order is preserved.
- Task 2: Construct a step-by-step trace table detailing pointer ~~int~~ iteration and condition evaluations when inserting val = 6 into the list.

Solve problem 4

1.25 Algorithm SortedInsert(head, val):
newNode ← CreateNode(val)
if head is Null or val < head.data then
newNode.next ← head
return newNode
end if
curr ← head
while curr.next is not Null and curr.next.data < val do:
curr ← curr.next
end while

Vibook
"Nó khiến người ta phát ra ánh sáng không phải là châu ngọc trên quần áo
mà là trí tuệ sâu trong tâm hồn".
Scanned with CamScanner

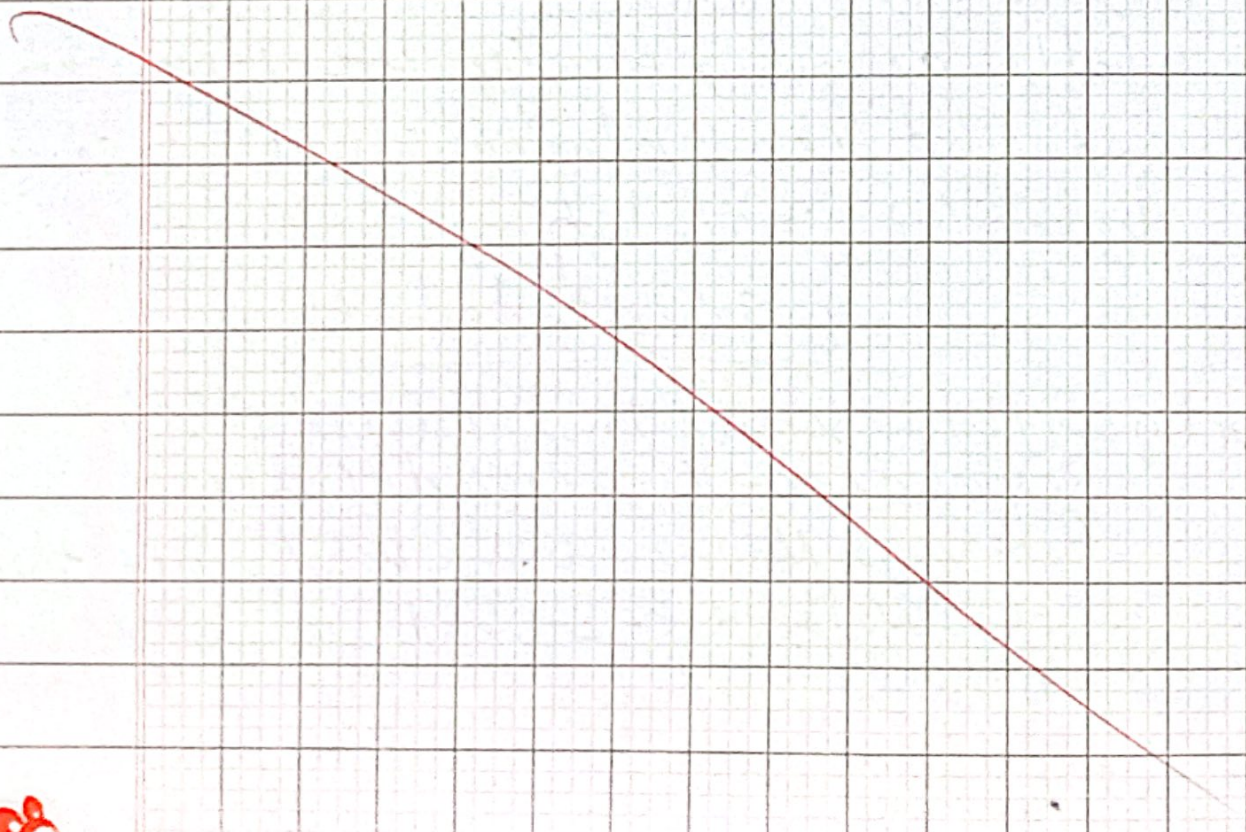
newNode.next ← curr.next
curr.next ← newNode
Thứ ngày tháng năm
return head

1.25 task 2:

Iteration	Condition Evaluation	Result
i=0	6 < 2	False
i=1	! = Null and 5<6	True
i=2	! = Null and 8<6	False

Final Result

head → 2 → 5 → 6 → 8 → Null



Scanned with CamScanner